Custom Built Operating System for Low Latency Message Handling in High-Frequency Trading

Rohit Suryadevara

1

suryader@mcmaster.ca

In Prototype 1.0, I set out to demonstrate that a purpose built, bare-metal trading kernel can achieve low microsecond decision-to-trade latency for a single security (AAPL) in emulation. By eliminating traditional sources of latency such as interrupts, syscalls, and context switches. Leveraging a DMA-driven receive path, cache-resident data structures, and a tight spin-poll loop, this prototype achieves performance on par with or exceeding advanced Linux-based systems.

1. Key Features of the Project

- **Bare-Metal Bootloading**: A custom 32-bit loader transitions from real mode to protected mode, sets up identity mapping, and hands off to a flat C runtime with minimal overhead.
- **MoldUDP64 Generator**: A configurable Python script simulates real-time UDP market data using MoldUDP64 formatting, emitting one initial quote followed by a Gaussian-random walk price stream among other features.
- **MoldUDP64 RTL NIC Parser**: Verilog FSM mimics SmartNIC behavior under QEMU/DPI, parsing payloads from AXI streams and signaling packet completion via MMIO writes. Decodes session, length, and payload fields.
- **Direct MMIO Reads & Writes**: Inline x86 assembly instructions perform memory-mapped NIC register reads and writes directly from C, bypassing all abstraction layers. Thus eliminating call overhead.
- **Reduction in Emulation (KVM/QEMU) Overhead**: Transitioning from QEMU's software translation (TCG) to hardware-assisted virtualization (KVM) greatly improves execution speed for the guest OS. This significantly reduces instruction emulation overhead and improves the accuracy of cycle-level measurements for kernel performance and packet processing.
- **Hugepages (Pre-allocated Guest RAM)**: The entire guest RAM region is allocated using '/dev/hugepages', eliminating page faults and reducing TLB pressure during high-frequency memory access.
- **Isolated Single-Core Execution**: The virtual CPU is pinned to a dedicated physical core using isolcpus and taskset on the host. This prevents the host OS from scheduling other tasks or migrating the guest thread, eliminating external interference and timing jitter — ensuring consistent cache residency and CPU cycle behavior
- **Hyperthreading Disabled**: Symmetric multithreading (SMT) is turned off to eliminate contention on shared L1/L2 caches and execution units.
- **Overclocking & Power-State Locking CPU**: The core runs at its highest stable frequency with deep C-states disabled to prevent frequency transitions.

- **Compiler-Level Tuning**: Profile-guided optimization, link-time optimization, and architecture-specific flags (‘-O3 -march=native‘) to eliminate overhead by avoiding unnecessary branches and maximizing IPC.
- **Cache-Line Prefetching & Isolation**: Descriptor rings are aligned to 64-byte boundaries prefaulted into L1, with head and tail indices placed on separate cache lines to reduce false sharing and improve coherence behavior.
- **All Logic L2-Resident**: The entirety of the decision path is constructed to fit within L2 cache, avoiding DRAM access during calling, parsing, and trade decision.
- **Data-Path Tuning**: Coded the Kernel with included features such as inline ASM, Manual loop unrolling and branchless techniques minimize branch mispredictions and expensive modulo operations.
- **TSC Timestamping**: Inline ‘rdtsc‘ and ‘rdtscp‘ instructions surround key operations (e.g., MMIO reads, decision logic, TX emission), enabling cycle-accurate latency measurement.
- **Debugging Tools**: An early UART console carries diagnostic prints—tagged for easy removal in production builds.

2. Prototype 2 Scope

Prototype 2.0 will introduce multi-symbol support (10+ instruments concurrently), a dual-CPU architecture with a secondary core running a custom built lean AI model for real-time strategy tuning, and an improved polling pipeline with additional architectural constraints to preserve deterministic cache residency. The AI engine will observe and modify strategy parameters live, implementing micro-adjustments for risk control and profit maximization.